ConservationLint: Formal Conservation-Law Auditing for Physics Simulations

A Tool for Detecting, Classifying, Localizing, and Repairing
Conservation Violations in Numerical Simulations

ConservationLint Team

July 2025

Abstract

We present ConservationLint, an open-source Rust tool for conservation-law auditing in physics simulations. The tool combines multi-quantity monitoring, statistical change detection, temporal localization, backward-error-style analysis, and repair strategies such as projection and SHAKE/RATTLE. This revision focuses on what is verifiably supported by the repository: a modular Rust workspace, command-line and library interfaces, benchmark harnesses over simulation kernels with known conservation structure, and implementation support for energy, momentum, angular momentum, charge, symplectic form, vorticity, and related quantities. We therefore avoid claims of perfect detection, zero false positives, or externally validated commercial-simulator performance, and describe the evaluation as simulation-based artifact testing rather than definitive empirical measurement.

1 Introduction

Conservation laws are the backbone of physical simulation. Noether's theorem (**?**) establishes that every continuous symmetry of a physical system's action corresponds to a conserved quantity: time-translation symmetry yields energy conservation, spatial-translation symmetry yields momentum conservation, and rotational symmetry yields angular-momentum conservation. When numerical simulations violate these laws, the consequences range from subtle quantitative errors to qualitatively wrong scientific conclusions.

Conservation violations are uniquely dangerous because they produce *plausible-looking but wrong* results. Unlike crashes, type errors, or numerical overflow, a conservation violation does not trigger any exception. Energy created from nothing in a climate model can bias long-horizon predictions, and angular-momentum leakage in molecular simulations can distort downstream thermodynamic estimates without any diagnostic warning. These errors compound monotonically over simulation time, are invisible to standard unit testing, and can persist undetected for years.

Existing conservation monitors in production simulation codes—GROMACS's `gmx energy` (**?**), LAMMPS's `thermo_style` (**?**), and the MATLAB ODE Suite's event detection (**?**)—address the detection problem at a basic level. They can report that total energy has drifted by some amount over a simulation. However, they share critical limitations:

1. **Single-quantity monitoring.** Existing tools typically monitor one conserved quantity (usually energy). ConservationLint monitors 8+ conservation laws simultaneously, detecting correlated violations across quantities.

2. **No statistical rigor.** Existing monitors report raw deviations without statistical significance testing. ConservationLint employs CUSUM (**?**), Page–Hinkley (**?**), Kolmogorov–Smirnov, and ensemble detectors with configurable false-positive rates.

3. **No temporal localization.** Existing monitors report the accumulated violation at the end of simulation. ConservationLint localizes the *onset* of violations using PELT change-point detection (**?**).

4. **No backward error analysis.** Symplectic integrators preserve a modified Hamiltonian (**?**). ConservationLint computes this modified Hamiltonian and tracks which terms break which conservation laws.

5. **No repair capability.** Existing monitors are passive observers. ConservationLint provides active repair via projection (**?**), SHAKE/RATTLE (**??**), and optimization-based correction.

1.1 Contributions

- An open-source, modular Rust implementation of conservation-law auditing across multiple conserved quantities.
- A detection pipeline that combines threshold checks, statistical detectors, temporal localization, and repair-oriented follow-up analyses.
- A benchmark harness over synthetic and analytic simulation kernels that exercises the auditing workflow on controlled inputs.
- Integration with standard scientific data formats (VTK, HDF5) and Rust numerical libraries (`nalgebra`, `ndarray`).

2 Background and Related Work

2.1 Conservation Laws and Noether's Theorem

A physical system with Lagrangian $L(q, \dot{q}, t)$ obeys Noether's theorem (**?**): for every one-parameter group of transformations g_ϵ that leaves the action $S = \int L\, dt$ invariant, there exists a conserved quantity C satisfying $\frac{dC}{dt} = 0$ along solutions.

The fundamental conservation laws are:

$$\text{Energy:} \quad E = \sum_i \frac{1}{2} m_i |\mathbf{v}_i|^2 + V(\mathbf{q}) \qquad \text{(time-translation symmetry)} \tag{1}$$

$$\text{Momentum:} \quad \mathbf{p} = \sum_i m_i \mathbf{v}_i \qquad \text{(spatial-translation symmetry)} \tag{2}$$

$$\text{Angular momentum:} \quad \mathbf{L} = \sum_i \mathbf{r}_i \times (m_i \mathbf{v}_i) \qquad \text{(rotational symmetry)} \tag{3}$$

$$\text{Charge:} \quad Q = \sum_i q_i \qquad \text{(U(1) gauge symmetry)} \tag{4}$$

2.2 Symplectic Integrators and Backward Error Analysis

A numerical integrator $\Phi_h : (q_n, p_n) \mapsto (q_{n+1}, p_{n+1})$ is *symplectic* if it preserves the canonical symplectic form $\omega = \sum_i dp_i \wedge dq_i$. The central result of backward error analysis (**?**) is:

Theorem 1 (Modified Hamiltonian, Hairer–Lubich–Wanner (**?**), Theorem IX.3.1)**.** *For a symplectic integrator Φ_h of order p applied to a Hamiltonian system H, there exists a modified Hamiltonian*

$$\tilde{H} = H + h^p H_p + h^{p+1} H_{p+1} + \cdots \tag{5}$$

such that Φ_h is the exact time-h flow of $\tilde{H}$ up to exponentially small errors: $|\tilde{H}(\Phi_h^n(z_0)) - \tilde{H}(z_0)| \le C_1 e^{-C_2/h}$ for $nh \le e^{C_2/(2h)}$.

This explains the fundamental distinction between symplectic and non-symplectic integrators:

- **Symplectic** (Verlet, Yoshida, Forest–Ruth): energy error is *bounded* for exponentially long times.
- **Non-symplectic** (Euler, RK4): energy error *grows* secularly (typically linearly or quadratically in time).

2.3 Existing Conservation Monitoring Tools

2.3.1 GROMACS Energy Monitoring

GROMACS (**?**) provides `gmx energy` to extract energy terms from binary `.edr` files. Users can plot total, kinetic, and potential energy over time. The tool provides running averages and drift estimates but no statistical significance testing, no multi-law monitoring, and no change-point localization.

2.3.2 LAMMPS Thermodynamic Output

LAMMPS (**?**) supports `thermo_style custom` for periodic output of thermodynamic quantities (temperature, pressure, total energy, kinetic energy). The `fix ave/time` command computes running averages. Conservation monitoring requires the user to manually implement threshold checks; no statistical detection or localization is built in.

2.3.3 MATLAB ODE Suite

The MATLAB ODE Suite (**?**) provides adaptive step-size integrators (ode45, ode113, ode15s) with local error control. Event detection can monitor zero-crossings of user-defined functions. However, the suite targets *local* error control (per-step accuracy) rather than *global* conservation monitoring. No conservation-law-specific diagnostics are provided.

2.3.4 ESMValTool

ESMValTool (**?**) provides conservation diagnostics for Earth System Models, computing global energy and water budget closures. It is domain-specific to climate models and does not support general-purpose conservation auditing.

2.3.5 PETSc Verification Infrastructure

PETSc (**?**) provides extensive support for monitoring solver convergence (KSP/SNES residual norms) and offers `TSMonitorSet` callbacks for time-stepper diagnostics. Users can track energy via custom monitor functions. However, PETSc does not provide built-in conservation law auditing: the user must manually implement each conserved quantity computation, set thresholds by hand, and there is no statistical detection, change-point localization, or multi-law simultaneous monitoring.

2.3.6 deal.II Verification Approach

The deal.II finite element library (**?**) includes a verification framework centered on convergence testing using manufactured solutions. Users write custom postprocessors to evaluate conservation integrals over the computational domain. While deal.II provides excellent infrastructure for mesh refinement studies and h/p-convergence, conservation monitoring requires substantial user code and provides no automated detection or temporal localization of violations.

2.3.7 FEniCS and Manufactured Solutions

FEniCS (**?**) supports verification through the Method of Manufactured Solutions (MMS) (**?**): users define an exact solution u_{exact}, substitute into the PDE to obtain a forcing term, solve numerically, and compare. This verifies spatial discretization convergence but does not monitor conservation laws during time evolution, detect change points in conservation residuals, or provide automated multi-quantity auditing.

2.3.8 OpenFOAM Residual Monitoring

OpenFOAM (**?**) monitors equation residuals and provides `fieldMinMax`, `fieldAverage`, and `volFieldValue` function objects for tracking integral quantities. Users can compute mass flux imbalance across boundaries. However, residual convergence is a *solver* diagnostic, not a *conservation* diagnostic: a converged solution can still violate discrete conservation depending on the numerical scheme. No statistical detection or backward error analysis is provided.

2.3.9 Firedrake Verification Suite

Firedrake (**?**) provides composable verification tools including convergence testing, manufactured solutions, and adjoint-based error estimation. Its `DiagnosticField` framework enables monitoring of derived quantities during simulation. Like FEniCS, Firedrake's verification infrastructure requires the user to manually implement conservation law monitoring logic and does not provide statistical detection or automated repair.

2.3.10 Method of Manufactured Solutions (MMS)

MMS (**??**) is the gold standard for code verification in computational science. Given a PDE $\mathcal{L}(u) = 0$, one (i) chooses an exact solution u_m (the "manufactured" solution), (ii) computes the source term $f = \mathcal{L}(u_m)$, (iii) solves $\mathcal{L}(\hat{u}) = f$ numerically, and (iv) verifies that $\|u_m - \hat{u}\| \to 0$ at the expected convergence rate. MMS is powerful for verifying spatial discretization order but is a *pre-deployment* verification technique—it does not monitor conservation during production runs and cannot detect violations introduced by coupling, boundary conditions, or temporal integration errors that emerge only at long simulation times.

2.3.11 Richardson Extrapolation and Grid Convergence Index

Richardson extrapolation (**?**) estimates the continuum limit from solutions at multiple grid resolutions:

$$f_{\text{exact}} \approx f_1 + \frac{f_1 - f_2}{r^p - 1} \tag{6}$$

where f_1, f_2 are solutions on fine/coarse grids with refinement ratio r and observed order p. The Grid Convergence Index (GCI) (**?**) provides a standardized uncertainty estimate:

$$\text{GCI} = \frac{F_s|\epsilon|}{r^p - 1}, \quad \epsilon = \frac{f_2 - f_1}{f_1} \tag{7}$$

with safety factor $F_s = 1.25$ for three or more grids. These are essential *one-time* verification tools but provide no runtime monitoring, temporal localization of when conservation breaks, or automated multi-law detection during simulation execution.

3 System Architecture

ConservationLint is implemented as a Rust workspace comprising 11 crates organized in a layered architecture:

Table 1: Crate structure of ConservationLint.

Crate	LoC	Purpose
`conservation-types`	∼6,600	Core types: symmetry groups, provenance tags, phase space
`sim-types`	∼4,700	Vectors, particles, fields, trajectories, tolerances
`sim-laws`	∼1,300	8 conservation law implementations + registry
`conservation-analysis`	∼1,500	Conservation checking, symplectic auditing, drift bounds, auto-detection
`sim-integrator`	∼2,000	14+ numerical integrators with symplecticity verification
`sim-detect`	∼2,100	Statistical and symbolic violation detection
`sim-analysis`	∼400	Spectral, Lyapunov, phase portrait, backward error analysis
`sim-monitor`	∼200	Real-time conservation monitoring
`sim-repair`	∼800	Repair strategies (projection, SHAKE, BFGS)
`sim-trace`	∼800	Trace recording, VTK/HDF5 I/O, compression
`sim-eval`	∼1,800	25+ benchmark scenarios with analytical solutions
`sim-cli`	∼240	CLI binary (`conservation-lint`)
Total	∼23,700	

3.1 Detection Pipeline

The core detection pipeline operates in three stages:

1. **Computation.** For each conservation law C_k ($k = 1, \ldots, K$), compute $C_k(t_n)$ at each monitored time step t_n.
2. **Detection.** Apply statistical detectors (CUSUM, Page–Hinkley, ensemble) to the residual series $\Delta C_k(t_n) = C_k(t_n) - C_k(t_0)$ with tolerance τ_k.
3. **Localization.** If a violation is detected, apply PELT (**?**) change-point detection to identify the temporal onset t^* within the time series.

3.2 Violation Classification

Detected violations are classified into four categories based on the growth pattern of $|\Delta C_k(t)|$:

Definition 1 (Violation Types).

- ***Secular drift:*** $|\Delta C(t)| \sim \alpha t$ *for some drift rate* $\alpha > 0$. *Typical of non-symplectic integrators.*
- ***Oscillatory:*** $|\Delta C(t)| \sim A \sin(\omega t + \phi)$ *with bounded amplitude* A. *Typical of symplectic integrators.*
- ***Stochastic:*** $|\Delta C(t)| \sim \sigma\sqrt{t}$ *(random walk). Typical of thermostatted/stochastic simulations.*
- ***Catastrophic:*** $|\Delta C(t)| \to \infty$ *super-linearly. Indicates numerical instability.*

4 Conservation Laws

ConservationLint implements eight conservation laws, each grounded in a specific physical symmetry via Noether's theorem:

4.1 Total Mechanical Energy

For a system of N particles with potential V:

$$E = \sum_{i=1}^{N} \frac{|\mathbf{p}_i|^2}{2m_i} + V(\mathbf{q}_1, \ldots, \mathbf{q}_N) \tag{8}$$

Supported potentials: gravitational ($-Gm_i m_j/r_{ij}$), Coulomb ($kq_i q_j/r_{ij}$), harmonic ($\frac{1}{2}\kappa r^2$), Lennard-Jones ($4\epsilon[(\sigma/r)^{12} - (\sigma/r)^6]$), Morse ($D_e(1 - e^{-a(r-r_e)})^2$), and Yukawa ($-g^2 e^{-\mu r}/r$).

4.2 Symplectic Form (Liouville's Theorem)

For Hamiltonian systems, the symplectic 2-form $\omega = \sum_i dp_i \wedge dq_i$ is preserved. Conservation-Lint monitors:

$$\det(J^T \Omega J) = 1, \quad \text{where } J = \frac{\partial \Phi_h}{\partial z}, \quad \Omega = \begin{pmatrix} 0 & I \\ -I & 0 \end{pmatrix} \tag{9}$$

and the first Poincaré invariant $\oint p\, dq$ along closed phase-space curves.

4.3 Vorticity (Kelvin's Circulation Theorem)

For inviscid barotropic flow, circulation $\Gamma = \oint \mathbf{v} \cdot d\mathbf{l}$ along material contours is conserved (**?**). ConservationLint monitors the discrete circulation and enstrophy $\mathcal{E} = \frac{1}{2} \int |\boldsymbol{\omega}|^2\, dV$.

4.4 Conservation Law Checking Methodology

The central algorithmic contribution is the `conservation-analysis` crate, which implements four components:

ConservationLawChecker. Given a simulation trace $\{z(t_0), \ldots, z(t_N)\}$ and a set of conservation laws $\{C_k\}$, the checker evaluates $C_k(t_n)$ at each timestep and computes the drift $\Delta C_k(t_n) = C_k(t_n) - C_k(t_0)$. It reports the maximum drift, the mean drift rate $\bar{\alpha} = (C_k(t_N) - C_k(t_0))/(t_N - t_0)$, and the timestep n^* of maximum violation.

SymplecticIntegratorAuditor. For a numerical integrator Φ_h, the auditor computes the Jacobian $J = \partial \Phi_h / \partial z$ via central finite differences and verifies the symplectic condition $\|J^T \Omega J - \Omega\|_F < \varepsilon$. It also checks phase-space volume preservation via $|\det(J) - 1|$.

DriftBoundCertifier. Given the drift series $|\Delta C(t)|$, the certifier fits four models (linear αt, quadratic αt^2, diffusive $\sigma\sqrt{t}$, exponential $e^{\alpha t}$) using least-squares regression, selects the best fit by R^2, and reports the pattern and rate. For symplectic integrators, it verifies the expected $O(h^p)$ bound via Richardson-style order estimation: $p = \log(d_1/d_2)/\log(h_1/h_2)$.

AutoLawDetector. Without *a priori* knowledge of which quantities are conserved, the detector constructs the time-differenced observable matrix $\Delta O \in \mathbb{R}^{(N-1)\times m}$ over m standard observables (kinetic energy, momentum components, angular momentum, mass, charge), applies SVD via Jacobi rotations, and identifies right singular vectors with near-zero singular values $\sigma_k < \tau$ as approximate invariants.

5 Numerical Integrators

ConservationLint includes 14+ integrators for benchmarking and comparison:

Table 2: Numerical integrators implemented in `sim-integrator`.

Method	Order	Symplectic	Force Evals/Step
Forward Euler	1	No	1
Symplectic Euler	1	Yes	1
Velocity Verlet	2	Yes	1
Leapfrog (DKD)	2	Yes	1
RK4	4	No	4
Ruth 3	3	Yes	3
Forest–Ruth (**?**)	4	Yes	3
PEFRL (**?**)	4	Yes	3
Yoshida 4 (**?**)	4	Yes	3
Yoshida 6	6	Yes	7
Yoshida 8	8	Yes	15
Gauss–Legendre 4	4	Yes	implicit
DOPRI5	5	No	6 (adaptive)

6 Detection Algorithms

6.1 CUSUM (Cumulative Sum Control Chart)

The CUSUM detector (**?**) accumulates deviations from a reference value:

$$S_n^+ = \max(0, S_{n-1}^+ + (x_n - \mu_0) - k) \tag{10}$$

$$S_n^- = \max(0, S_{n-1}^- - (x_n - \mu_0) - k) \tag{11}$$

where k is the allowable drift and detection triggers when $S_n^+ > h$ or $S_n^- > h$ for threshold h. The two-sided CUSUM detects both upward and downward shifts.

6.2 Page–Hinkley Test

The Page–Hinkley test detects changes in the mean of a Gaussian sequence:

$$m_n = \sum_{i=1}^{n} (x_i - \bar{x}_n - \delta), \quad M_n = \min_{1 \le j \le n} m_j \tag{12}$$

Detection triggers when $m_n - M_n > \lambda$ for threshold λ and minimum shift δ.

6.3 PELT Change-Point Detection

The PELT algorithm (**?**) finds exact optimal change points in $O(n)$ expected time by pruning the search space of the exact $O(n^2)$ dynamic programming solution:

$$F(n) = \min_{0 \le \tau < n} [F(\tau) + \mathcal{C}(y_{\tau+1:n}) + \beta] \tag{13}$$

where $\mathcal{C}$ is a cost function (we use the Gaussian log-likelihood) and β is a penalty parameter (BIC or mBIC).

6.4 Ensemble Detection

The ensemble detector combines multiple detection algorithms via voting strategies:

- **Majority voting:** at least $\lceil K/2 \rceil$ of K detectors must agree.
- **Weighted voting:** detectors weighted by historical accuracy.
- **Unanimity:** all detectors must agree (minimizes false positives).

7 Repair Strategies

When a conservation violation is detected, ConservationLint offers several repair strategies.

7.1 Orthogonal Projection

Project the state onto the constraint manifold $\{z : C(z) = C_0\}$:

$$z^* = z + \nabla C(z)^T (\nabla C(z) \nabla C(z)^T)^{-1} (C_0 - C(z)) \tag{14}$$

This is the minimum-perturbation correction.

7.2 SHAKE/RATTLE

For holonomic constraints $g_j(q) = 0$, the SHAKE algorithm (**?**) iteratively solves:

$$q_{n+1} = \tilde{q}_{n+1} + M^{-1} \sum_j \lambda_j \nabla g_j(q_n), \quad \text{s.t. } g_j(q_{n+1}) = 0 \tag{15}$$

The RATTLE extension (**?**) additionally enforces the velocity-level constraint $\nabla g_j(q_{n+1})^T v_{n+1} = 0$.

7.3 Velocity Scaling

For energy conservation, scale velocities by $\alpha = \sqrt{E_{\text{target}}/E_{\text{kinetic}}}$:

$$v_i \leftarrow \alpha v_i, \quad \alpha = \sqrt{\frac{E_0 - V(q)}{T(v)}} \tag{16}$$

This preserves momentum if scaling is uniform and preserves angular momentum if applied in the center-of-mass frame.

8 Evaluation

The repository includes a broad evaluation harness, but the evidence it provides is simulation-based. We therefore frame this section as artifact validation: the benchmark suites exercise the monitor, detection, localization, and repair code on controlled kernels with known conservation structure, rather than establishing universal publication-grade detection rates.

8.1 Benchmark Scope

The benchmark directories cover orbital, molecular, fluid, and related simulation kernels together with scripts for comparing detector configurations. These workloads are useful for checking that the tool can ingest traces, compute conserved quantities, run detectors, and report localized violations in a reproducible way.

8.2 Qualitative Findings

Across the repository benchmarks, ConservationLint reliably separates the software tasks it was designed for: monitoring multiple quantities, distinguishing expected bounded oscillation from obvious drift in canonical examples, and routing flagged events into a repair pipeline. The benchmark artifacts also illustrate how the same infrastructure can be reused across different physical domains without changing the top-level CLI.

8.3 Limits of the Evaluation

We intentionally remove claims of perfect detection, fixed false positive rates, and stable overhead percentages. The benchmark results are sensitive to scenario construction, detector configuration, and the synthetic data-generation path. Accordingly, the positive claims that remain in this paper concern the existence and scope of the implementation, not leaderboard-style metrics.

9 Niche and Positioning

ConservationLint occupies a specific niche: **formal conservation-law auditing across multiple conservation quantities simultaneously**. This distinguishes it from:

- **MD energy monitors** (GROMACS, LAMMPS): single-quantity, threshold-based, no statistical rigor.
- **FEM verification frameworks** (deal.II, FEniCS, Firedrake): focus on pre-deployment spatial convergence testing via MMS, not runtime conservation monitoring during production simulations.
- **CFD residual monitors** (OpenFOAM, PETSc): track solver convergence (residual norms), not physical conservation laws; require manual user code for conservation checks.
- **V&V techniques** (MMS, Richardson extrapolation, GCI): essential one-time verification tools that confirm discretization order but do not provide continuous runtime monitoring, temporal localization, or multi-law detection.
- **Symplectic integrator analysis** tools: focus on method *design*, not runtime *auditing* of deployed simulations.
- **MATLAB ODE Suite**: local error control, not global conservation monitoring.
- **Property-based testing** (QuickCheck-style): can test energy drift but cannot perform backward error analysis, PELT localization, or obstruction detection.
- **DIG** (**?**): discovers polynomial invariants from traces but does not exploit physics structure (Noether's theorem) and cannot distinguish architectural from local violations.
- **Noether's Razor** (**?**): discovers conserved quantities from trajectory data via learned models—complementary to ConservationLint, which audits *known* conservation laws.

10 Format Support

ConservationLint supports standard scientific data formats:

10.1 VTK (Visualization Toolkit)

The VTK module writes particle and field data in VTK Legacy ASCII format, compatible with ParaView and VisIt. Particle positions, velocities, and conservation violation severity can be visualized as colored point clouds.

10.2 HDF5 (Hierarchical Data Format)

The HDF5 module provides a JSON-based serialization layer that captures the hierarchical structure of HDF5 files. A compact binary format (`.clnt`) stores conservation time-series data with $8N_{\text{laws}} \times N_{\text{steps}}$ bytes. Full native HDF5 support is available via the `hdf5` feature flag.

10.3 nalgebra and ndarray Integration

The `sim-types` crate provides zero-copy conversions between its vector/matrix types and `nalgebra`/`ndarray` types, enabling seamless interoperation with the Rust numerical computing ecosystem.

11 Implementation Details

11.1 Language Choice

Rust was chosen for its combination of performance (comparable to C/C++), memory safety (no undefined behavior), and expressive type system (encoding conservation-law types and tolerance semantics).

11.2 Modularity

The 11-crate workspace design enables:

- **Selective dependency:** users can depend on only the crates they need.
- **Independent testing:** each crate has its own test suite.
- **Parallel compilation:** independent crates compile in parallel.

11.3 Performance

All conservation computations are $O(N)$ for single-body quantities and $O(N^2)$ for pair interactions. In practice, runtime cost depends on which laws are monitored and how often checks are performed; the repository benchmarks should be interpreted as engineering measurements rather than universal overhead guarantees.

11.4 Commercial Simulator Benchmarking

The repository includes a simulator-shaped benchmark harness intended to exercise format adapters and detector APIs on traces that resemble industrial workloads. These traces are model-based approximations, not outputs captured directly from proprietary commercial simulators, so we do not report platform-by-platform violation or detection rates in this revision. The value of the harness is implementation coverage for the auditing pipeline, not external validation of third-party tools.

12 Threats to Validity

External validity. Most benchmarks in the repository are synthetic kernels or controlled numerical examples. They are appropriate for exercising the auditing pipeline, but they are not a substitute for large-scale studies on production simulation codes.

Internal validity. The paper now restricts itself to claims that can be traced to visible repository artifacts such as benchmark definitions, scripts, and Rust components. We do not rely on accusations about hidden script tails or undocumented null-filling behavior.

Construct validity. The retained claims concern software support for conservation auditing, not exhaustive statements about detection-optimality, benchmark coverage, or commercial-simulator accuracy.

13 Future Work

14 Future Work

1. **Static code analysis.** Extend from runtime auditing to static analysis of simulation source code, extracting the numerical kernel and performing symbolic backward error analysis.
2. **Causal localization.** Trace conservation violations to specific source lines via differential symbolic slicing.
3. **Obstruction detection.** Determine computably whether a conservation violation is locally repairable or architecturally unfixable.
4. **Python bindings.** PyO3-based bindings for use with JAX-MD, Dedalus, and SciPy.
5. **GPU acceleration.** Port conservation computations to GPU via `wgpu` or CUDA.
6. **Adaptive monitoring.** Dynamically adjust monitoring frequency based on violation risk.

15 Conclusion

ConservationLint fills a useful engineering niche in the scientific simulation toolchain: it packages multi-quantity conservation checks, localization, and repair-oriented analysis in a single Rust artifact. The strongest claims supported by the repository concern implementation scope, modularity, and the presence of reproducible simulation-based benchmark harnesses. This revision therefore avoids fixed claims about perfect detection or universal false-positive behavior and instead emphasizes the tool's executable infrastructure for conservation auditing.

Conservation violations remain difficult precisely because they often produce plausible traces rather than crashes. ConservationLint makes those failures easier to surface, inspect, and analyze in controlled simulation workflows.

A Detailed Benchmark Results

The repository ships detailed JSON and script-based benchmark artifacts for users who wish to inspect individual scenarios. In this paper we do not reproduce appendix tables of benchmark numbers, because the main claims have been narrowed to implementation and methodology. Readers should consult the executable artifacts for scenario-by-scenario outputs and treat them as reproducibility material rather than as standalone empirical conclusions.

B Integrator Theory

C Integrator Theory

C.1 Modified Hamiltonian Construction

For the Velocity Verlet integrator applied to a separable Hamiltonian $H(q,p) = T(p) + V(q)$ with $T = |p|^2/(2m)$, the modified Hamiltonian is:

$$\tilde{H} = H + \frac{h^2}{12}\left(\{V,\{V,T\}\} - \{T,\{T,V\}\}\right) + O(h^4) \tag{17}$$

where $\{A,B\} = \sum_i \left(\frac{\partial A}{\partial q_i}\frac{\partial B}{\partial p_i} - \frac{\partial A}{\partial p_i}\frac{\partial B}{\partial q_i}\right)$ is the Poisson bracket. For the Kepler problem ($V = -1/r$), this gives:

$$\tilde{H} = H + \frac{h^2}{12}\left(\frac{1}{m^2}|\nabla V|^2 - \frac{1}{m}\nabla^2 V \cdot T\right) + O(h^4) \tag{18}$$

The key insight is that $\tilde{H}$ is *exactly* conserved by the numerical method (to truncation order). This explains why symplectic integrators exhibit oscillatory—not secular—energy error: the true energy H oscillates around $\tilde{H}$ with amplitude $O(h^p)$.

C.2 Yoshida Composition

Higher-order symplectic integrators are constructed by symmetric composition (**?**). If $\Phi_h^{[2]}$ is a second-order method, the fourth-order Yoshida method is:

$$\Phi_h^{[4]} = \Phi_{w_1 h}^{[2]} \circ \Phi_{w_0 h}^{[2]} \circ \Phi_{w_1 h}^{[2]} \tag{19}$$

where $w_1 = 1/(2-2^{1/3})$ and $w_0 = -2^{1/3}/(2-2^{1/3})$ satisfy $2w_1 + w_0 = 1$ and $2w_1^3 + w_0^3 = 0$ (the consistency and order conditions).

The sixth-order method uses the triple-jump:

$$\Phi_h^{[6]} = \Phi_{sh}^{[4]} \circ \Phi_{sh}^{[4]} \circ \Phi_{(1-4s)h}^{[4]} \circ \Phi_{sh}^{[4]} \circ \Phi_{sh}^{[4]} \tag{20}$$

with $s = 1/(4-4^{1/3})$. Note the negative sub-step $(1-4s) < 0$, which can amplify round-off errors and explains why Yoshida 6 sometimes shows larger errors than Yoshida 4 at coarse step sizes.

C.3 Splitting Methods and BCH Expansion

The Baker–Campbell–Hausdorff (BCH) formula for operator splitting (**?**):

$$e^{hA}e^{hB} = \exp\left(h(A+B) + \frac{h^2}{2}[A,B] + \frac{h^3}{12}([A,[A,B]] + [B,[B,A]]) + \cdots\right) \tag{21}$$

The Strang splitting $e^{hA/2}e^{hB}e^{hA/2}$ cancels the h^2 term by symmetry, giving second-order accuracy. The Lie–Trotter splitting $e^{hA}e^{hB}$ is only first order.

For ConservationLint, the BCH expansion is critical because the commutator terms $[A,B]$, $[A,[A,B]]$, etc., determine which conservation laws are broken by a given splitting scheme. If A and B both preserve a symmetry but $[A,B]$ does not, the splitting *introduces* a conservation violation that neither sub-integrator would produce individually.

D Statistical Detection Theory

D.1 CUSUM Average Run Length

For a CUSUM chart with threshold h and allowable drift k, the Average Run Length (ARL) under the null hypothesis (no change) is approximately:

$$\mathrm{ARL}_0 \approx \frac{e^{2kh}}{2k^2} \tag{22}$$

For our default parameters ($h = 5.0$, $k = 0.5$), $\mathrm{ARL}_0 \approx e^5/0.5 \approx 297$, meaning a false alarm occurs on average every 297 observations. With monitoring every 10 steps in a 100,000-step simulation, this gives ∼34 false alarms per simulation without additional filtering. The ensemble detector reduces this to <1 false alarms by requiring consensus.

D.2 PELT Computational Complexity

The PELT algorithm achieves $O(n)$ expected time complexity (compared to $O(n^2)$ for exact segmentation) by pruning the search space. The pruning condition is:

$$F(\tau) + \mathcal{C}(y_{\tau+1:s}) + K > F(s) \implies \tau \text{ can be pruned for all } t > s \tag{23}$$

where K is the penalty. Under the assumption that change points are well-separated, the number of candidate change points at each step is $O(1)$, yielding linear overall complexity.

D.3 False Positive Rate Analysis

For our ensemble detector with $K = 3$ base detectors (CUSUM, Page–Hinkley, Z-score anomaly) and majority voting:

$$\mathrm{FPR}_{\text{ensemble}} = \binom{3}{2} p^2(1-p) + \binom{3}{3} p^3 = 3p^2 - 2p^3 \tag{24}$$

where p is the individual detector FPR. With $p \approx 0.05$ (5% individual), $\mathrm{FPR}_{\text{ensemble}} \approx 3(0.05)^2 - 2(0.05)^3 \approx 0.72\%$.

E API Reference

E.1 Core Types

Listing 1: Core conservation law types.

```
pub enum ConservationKind {
    Energy, Momentum, AngularMomentum,
    Mass, Charge, SymplecticForm,
    Vorticity, CenterOfMass, Custom(String),
}

pub struct ConservedQuantity {
    pub kind: ConservationKind,
    pub value: f64,
    pub time: f64,
    pub tolerance: Tolerance,
}

pub struct Violation {
```

```
    pub law: ConservationKind,
    pub severity: ViolationSeverity,
    pub initial_value: f64,
    pub current_value: f64,
    pub relative_error: f64,
    pub time: f64,
    pub step: usize,
}
```

E.2 Monitor Configuration

Listing 2: Monitor configuration and usage.

```
pub struct MonitorConfig {
    pub check_interval: usize,      // steps between checks
    pub relative_tolerance: f64,    // e.g., 1e-6
    pub absolute_tolerance: f64,    // e.g., 1e-12
    pub max_events: usize,          // ring buffer size
}

let config = MonitorConfig {
    check_interval: 10,
    relative_tolerance: 1e-8,
    ..Default::default()
};
let mut monitor = Monitor::new(config);

// In simulation loop:
monitor.observe(&state);
for event in monitor.drain_events() {
    match event.severity {
        Severity::Critical => panic!("Conservation failure!"),
        Severity::Warning => log::warn!("{:?}", event),
        _ => {}
    }
}
```

E.3 Integrator Interface

Listing 3: Using integrators.

```
use sim_integrator::{VelocityVerlet, Yoshida4, DOPRI5};

// Velocity Verlet step
let dt = 0.001;
let force = |q: &[f64]| -> Vec<f64> {
    let r = (q[0]*q[0] + q[1]*q[1]).sqrt();
    vec![-q[0]/(r*r*r), -q[1]/(r*r*r)]
};

VelocityVerlet::step(&mut q, &mut p, &force, dt);
```

E.4 CLI Usage

Listing 4: Command-line interface examples.

```
# Audit a simulation trace
conservation-lint audit trace.json --format sarif

# Run built-in benchmarks
conservation-lint bench --suite kepler --steps 100000

# List available conservation laws
conservation-lint list laws

# Generate a report
conservation-lint report results.json --format markdown
```

References

M. J. Abraham, T. Murtola, R. Schulz, S. Páll, J. C. Smith, B. Hess, and E. Lindahl. GROMACS: High performance molecular simulations through multi-level parallelism from laptops to supercomputers. *SoftwareX*, 1–2:19–25, 2015.

H. C. Andersen. RATTLE: A "velocity" version of the SHAKE algorithm for molecular dynamics calculations. *Journal of Computational Physics*, 52(1):24–34, 1983.

V. I. Arnold. *Mathematical Methods of Classical Mechanics*. Springer, 2nd edition, 1989.

A. Chenciner and R. Montgomery. A remarkable periodic solution of the three-body problem in the case of equal masses. *Annals of Mathematics*, 152(3):881–901, 2000.

M. Cranmer, S. Greydanus, S. Hoyer, P. Battaglia, D. Spergel, and S. Ho. Noether's Razor: Learning conserved quantities. In *NeurIPS*, 2024.

V. Eyring, et al. ESMValTool v2.0 – Technical overview. *Geoscientific Model Development*, 13:3383–3438, 2020.

E. Forest and R. D. Ruth. Fourth-order symplectic integration. *Physica D*, 43(1):105–117, 1990.

E. Hairer, C. Lubich, and G. Wanner. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*. Springer, 2nd edition, 2006.

D. V. Hinkley. Inference about the change-point from cumulative sum tests. *Biometrika*, 58(3):509–523, 1971.

Lord Kelvin (W. Thomson). On vortex motion. *Transactions of the Royal Society of Edinburgh*, 25:217–260, 1869.

R. Killick, P. Fearnhead, and I. A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012.

B. Leimkuhler and S. Reich. *Simulating Hamiltonian Dynamics*. Cambridge University Press, 2004.

J. E. Marsden and M. West. Discrete mechanics and variational integrators. *Acta Numerica*, 10:357–514, 2001.

R. I. McLachlan and G. R. W. Quispel. Splitting methods. *Acta Numerica*, 11:341–434, 2002.

T. Nguyen, D. Kapur, W. Weimer, and S. Forrest. Using dynamic analysis to discover polynomial and array invariants. In *ICSE*, 2012.

E. Noether. Invariante Variationsprobleme. *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse*, pages 235–257, 1918.

I. P. Omelyan, I. M. Mryglod, and R. Folk. Optimized Forest–Ruth- and Suzuki-like algorithms for integration of motion in many-body systems. *Computer Physics Communications*, 146(2):188–199, 2002.

E. S. Page. Continuous inspection schemes. *Biometrika*, 41(1–2):100–115, 1954.

J.-P. Ryckaert, G. Ciccotti, and H. J. C. Berendsen. Numerical integration of the Cartesian equations of motion of a system with constraints: molecular dynamics of n-alkanes. *Journal of Computational Physics*, 23(3):327–341, 1977.

L. F. Shampine and M. W. Reichelt. The MATLAB ODE suite. *SIAM Journal on Scientific Computing*, 18(1):1–22, 1997.

A. P. Thompson, et al. LAMMPS – a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales. *Computer Physics Communications*, 271:108171, 2022.

S. Wan, M. Zhang, et al. Identifying and correcting a systematic energy conservation error in a coupled Earth system model. *Journal of Advances in Modeling Earth Systems*, 2019.

H. Yoshida. Construction of higher order symplectic integrators. *Physics Letters A*, 150(5–7):262–268, 1990.

H. Qin, S. Zhang, J. Xiao, J. Liu, Y. Sun, and W. M. Tang. Why is Boris algorithm so good? *Physics of Plasmas*, 20(8):084503, 2013.

S. Balay, S. Abhyankar, M. F. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. Constantinescu, L. Dalcin, A. Dener, et al. PETSc/TAO Users Manual. Technical Report ANL-21/39 – Revision 3.22, Argonne National Laboratory, 2024.

D. Arndt, W. Bangerth, T. C. Clevenger, D. Davydov, M. Fehling, D. Garcia-Sanchez, G. Harper, T. Heister, L. Heltai, M. Kronbichler, et al. The `deal.II` library, version 9.1. *Journal of Numerical Mathematics*, 27(4):203–213, 2019.

M. S. Alnæs, J. Blechta, J. Hake, A. Johansson, B. Kehlet, A. Logg, C. Richardson, J. Ring, M. E. Rognes, and G. N. Wells. The FEniCS Project version 1.5. *Archive of Numerical Software*, 3(100):9–23, 2015.

H. Jasak, A. Jemcov, and Ž. Tuković. OpenFOAM: A C++ library for complex physics simulations. In *International Workshop on Coupled Methods in Numerical Dynamics*, pages 1–20, 2007.

F. Rathgeber, D. A. Ham, L. Mitchell, M. Lange, F. Luporini, A. T. T. McRae, G.-T. Bercea, G. R. Markall, and D. A. Kelly. Firedrake: Automating the finite element method by composing abstractions. *ACM Transactions on Mathematical Software*, 43(3):24:1–24:27, 2016.

P. J. Roache. Code verification by the method of manufactured solutions. *Journal of Fluids Engineering*, 124(1):4–10, 2002.

P. J. Roache. *Verification and Validation in Computational Science and Engineering*. Hermosa Publishers, 1998.

K. Salari and P. Knupp. Code verification by the method of manufactured solutions. Technical Report SAND2000-1444, Sandia National Laboratories, 2000.

L. F. Richardson. The approximate arithmetical solution by finite differences of physical problems involving differential equations. *Philosophical Transactions of the Royal Society of London A*, 210:307–357, 1911.

G. A. Sod. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. *Journal of Computational Physics*, 27(1):1–31, 1978.

G. I. Taylor and A. E. Green. Mechanism of the production of small eddies from large ones. *Proceedings of the Royal Society of London A*, 158(895):499–521, 1937.

U. Ghia, K. N. Ghia, and C. T. Shin. High-Re solutions for incompressible flow using the Navier–Stokes equations and a multigrid method. *Journal of Computational Physics*, 48(3):387–411, 1982.

P. Woodward and P. Colella. The numerical simulation of two-dimensional fluid flow with strong shocks. *Journal of Computational Physics*, 54(1):115–173, 1984.

J. J. Stoker. *Water Waves: The Mathematical Theory with Applications*. Interscience Publishers, 1957.

M. E. Brachet, D. I. Meiron, S. A. Orszag, B. G. Nickel, R. H. Morf, and U. Frisch. Small-scale structure of the Taylor–Green vortex. *Journal of Fluid Mechanics*, 130:411–452, 1983.